

Sim. Nesse caso, eu faria **um segundo resumo diferente do anterior**: não seria apenas uma revisão da apostila, mas um “**Resumo para quem nunca abriu o Livro de Estudos**”.

A ideia é que você consiga começar do zero e chegar ao nível necessário para entender os simulados. Vou manter a estrutura do CTFL e dar mais explicação aos conceitos, principalmente onde o material anterior estava mais “telegráfico”. O próprio livro define objetivos de aprendizagem K1, K2 e K3, e os simulados cobram bastante aplicação, especialmente nos capítulos 4 e 5.

RESUMO CTFL — DO ZERO ATÉ A PROVA

Como usar este resumo

O CTFL não é simplesmente uma lista de definições para decorar.

Você precisa aprender três coisas:

K1 — lembrar

Saber reconhecer ou recordar um conceito.

K2 — compreender

Conseguir explicar o conceito e reconhecer quando ele está sendo usado.

K3 — aplicar

Receber uma situação e conseguir aplicar o conceito ou técnica.

Isso é especialmente importante nos capítulos 4 e 5. Os simulados têm 40 questões e concentram **20 questões nos capítulos 4 e 5**, além de apresentarem muitos cenários para você analisar.

CAPÍTULO 1 — FUNDAMENTOS DE TESTE

1. O que é teste?

Teste de software é um **conjunto de atividades usadas para descobrir defeitos e avaliar a qualidade de produtos de software.**

Um erro comum é pensar:

“Testar é abrir o sistema, clicar nos botões e ver se funciona.”

Isso é apenas uma parte do teste.

O teste pode envolver:

- analisar requisitos;
- revisar documentos;
- criar condições de teste;
- criar casos de teste;
- preparar dados;
- preparar ambiente;
- executar testes;
- analisar resultados;
- registrar defeitos;
- produzir relatórios;
- avaliar riscos.

O teste também ajuda a **reduzir o risco de falhas em operação.**

Objeto de teste

É aquilo que está sendo testado.

Pode ser:

- um requisito;
- uma especificação;
- um componente;
- uma API;
- uma tela;
- um sistema completo;
- uma documentação;
- um modelo.

2. Verificação × Validação

Essa diferença é muito importante.

Verificação

Pergunta:

“Estamos construindo o produto corretamente?”

Verifica se o produto está de acordo com requisitos e especificações.

Exemplo:

O requisito diz:

“A senha deve possuir pelo menos 8 caracteres.”

A verificação procura saber se o sistema atende a essa especificação.

Validação

Pergunta:

“Estamos construindo o produto certo?”

Verifica se o produto realmente atende às necessidades dos usuários e stakeholders.

Exemplo:

O sistema pode cumprir perfeitamente todos os requisitos escritos, mas ser extremamente difícil de usar.

Nesse caso, ele pode estar correto segundo a especificação, mas não atender adequadamente à necessidade do usuário.

O CTFL trata teste como envolvendo tanto verificação quanto validação.

3. Teste × Depuração

Não são a mesma coisa.

Teste

Procura **revelar problemas**.

Exemplo:

O tester executa o login com uma senha inválida e percebe que o sistema permite acesso.

Foi encontrado um comportamento incorreto.

Depuração

Procura **descobrir a causa do problema e corrigi-lo**.

Fluxo simplificado:

Teste

↓

encontra falha

↓

Depuração

↓

investiga a causa

↓

corrige o código

↓

Novo teste

↓

verifica se a correção funcionou

Então:

Tester encontra/revela o problema.

Desenvolvedor realiza a depuração para encontrar e corrigir a causa.

4. Erro → Defeito → Falha

Essa sequência precisa estar muito clara.

Erro

É uma ação humana incorreta.

Exemplo:

O desenvolvedor deveria escrever:

`>= 18`

mas escreveu:

`> 18`

O desenvolvedor cometeu um **erro**.

Defeito

O erro introduz um problema no software.

Nesse caso, o código contém uma condição incorreta.

Isso é um **defeito**.

Falha

Quando o software é executado e o defeito produz um comportamento incorreto, temos uma **falha**.

Por exemplo:

Uma pessoa com exatamente 18 anos tenta criar uma conta e o sistema rejeita.

Decore a cadeia:

Pessoa faz algo errado → ERRO

Erro fica no software → DEFEITO

Defeito é executado e produz comportamento incorreto → FALHA

5. Causa raiz

A causa raiz é a causa fundamental que está por trás de um problema.

Exemplo:

O sistema apresentou uma falha:

Cliente não conseguiu fazer login.

Investigação:

- login falhou;
- código recebeu senha incorretamente;
- formulário enviava campo errado;
- requisito tinha sido interpretado incorretamente.

A interpretação incorreta do requisito pode ser a **causa raiz**.

Encontrar a causa raiz é importante porque corrigir somente o sintoma pode fazer o problema voltar.

6. Por que precisamos testar?

Software pode apresentar problemas.

Esses problemas podem gerar:

- perda financeira;
- perda de tempo;
- perda de reputação;
- problemas para usuários;
- riscos de segurança;
- em sistemas críticos, danos graves.

O objetivo não é provar que:

“O software não possui nenhum defeito.”

Isso não é possível.

O teste fornece informações sobre a qualidade e ajuda a **reduzir riscos**.

7. QA × Teste

QA — Quality Assurance

Está relacionado à **garantia da qualidade** e à melhoria dos processos utilizados para produzir software.

A ideia é prevenir problemas.

Exemplo:

melhorar processos, padrões, práticas e formas de trabalho para reduzir a ocorrência de defeitos.

Teste

Tem foco em avaliar produtos e revelar problemas.

Uma forma simples de lembrar:

QA → prevenção e melhoria do processo

Teste → avaliação do produto e descoberta de problemas

Não são sinônimos.

8. Os 7 princípios de teste

Esses princípios são muito importantes.

1. Testes mostram a presença de defeitos

Testar pode encontrar defeitos.

Mas um teste que não encontrou nenhum defeito **não prova que não existem defeitos**.

2. Testes exaustivos são impossíveis

Não conseguimos testar absolutamente todas as combinações possíveis.

Imagine um sistema com:

- milhares de usuários;
- diversas configurações;
- diferentes navegadores;
- diferentes dispositivos;
- milhares de entradas.

Seria impossível testar todas as combinações.

Por isso utilizamos técnicas para selecionar testes relevantes.

3. Teste antecipado

Quanto mais cedo um problema for encontrado, menor tende a ser o custo para corrigi-lo.

Exemplo:

Encontrar um requisito errado durante a revisão é normalmente melhor do que descobrir o problema depois que todo o sistema foi desenvolvido.

Isso está relacionado ao conceito de **shift-left**.

4. Agrupamento de defeitos

Uma pequena quantidade de partes do sistema pode conter uma grande quantidade dos defeitos.

Por isso, quando encontramos muitos defeitos em determinada área, pode ser interessante concentrar testes nela.

5. Paradoxo do pesticida

Se repetirmos exatamente os mesmos testes muitas vezes, eles podem deixar de encontrar novos defeitos.

É necessário:

- revisar testes;
 - criar novos testes;
 - variar entradas;
 - utilizar técnicas diferentes.
-

6. Teste depende do contexto

Não existe uma única maneira de testar todos os sistemas.

Um:

- aplicativo bancário;
- jogo;
- sistema médico;
- site de notícias;
- sistema embarcado

possui riscos e necessidades diferentes.

A estratégia de teste deve considerar o contexto.

7. Ausência de erros é uma falácia

Mesmo que todos os testes planejados passem, o sistema pode continuar não atendendo às necessidades dos usuários.

Ou seja:

“Não encontrei defeitos” não significa “o produto é perfeito”.

9. Atividades de teste

O processo de teste pode ser entendido como várias atividades.

Uma visão geral:

Planejamento

↓

Monitoramento e controle

↓

Análise

↓

Modelagem

↓

Implementação

↓

Execução

↓

Conclusão

Planejamento

Decidir:

- o que será testado;
- como;
- quando;
- por quem;
- com quais recursos;
- quais riscos existem.

Análise

Pergunta:

O que precisa ser testado?

São identificadas condições de teste a partir da base de teste.

Modelagem

Pergunta:

Como vamos testar?

São criados casos de teste, por exemplo.

Implementação

Preparação dos elementos necessários para executar os testes.

Execução

Executar os testes e comparar:

resultado esperado × resultado obtido

Conclusão

Avaliar os resultados e finalizar as atividades.

10. Testware

Testware é o conjunto de produtos de trabalho produzidos ou utilizados durante o processo de teste.

Exemplos:

- plano de teste;
- casos de teste;
- condições de teste;
- dados de teste;
- procedimentos;
- scripts;
- registros;
- relatórios;
- critérios de teste;
- ambiente de teste.

Uma forma fácil de pensar

Se o processo de teste produz ou utiliza determinado artefato, ele pode fazer parte do **Testware**.

Não confunda:

Testware não significa “software para testar”.

É o conjunto de **produtos de trabalho relacionados ao teste**.

11. Rastreabilidade

Imagine:

Requisito

↓

Condição de teste

↓

Caso de teste

↓

Resultado

↓

Defeito

Essa ligação é chamada de **rastreabilidade**.

Ela permite responder:

“Esse requisito foi testado?”

Também ajuda a:

- avaliar cobertura;
 - analisar impacto de mudanças;
 - acompanhar riscos;
 - realizar auditorias;
 - compreender resultados.
-

12. Papéis no teste

Uma distinção importante:

Gerenciamento de teste

Principalmente:

- planejamento;
- monitoramento;
- controle;
- conclusão.

Testador

Principalmente:

- análise;
- modelagem;
- implementação;
- execução.

Isso não significa que uma pessoa nunca possa realizar atividades do outro grupo. É uma forma de diferenciar responsabilidades principais.

CAPÍTULO 2 — TESTES AO LONGO DO CICLO DE VIDA

Agora precisamos entender **quando e onde testar**.

13. SDLC

SDLC significa:

Software Development Life Cycle

ou

Ciclo de Vida de Desenvolvimento de Software.

É a forma como o software é desenvolvido e mantido.

O modelo escolhido influencia:

- quando testar;
- quais testes realizar;
- documentação;
- automação;
- responsabilidades;

- níveis de teste.
-

14. Modelos de desenvolvimento

Sequencial

Exemplos:

- Waterfall;
- V-Model.

As atividades tendem a acontecer em uma sequência mais definida.

Iterativo

O produto é desenvolvido em ciclos.

Exemplo:

- Spiral.

Incremental

O produto é entregue em partes.

Exemplo:

Primeiro:

login

Depois:

cadastro

Depois:

pagamentos

E assim por diante.

15. Testar não significa esperar o software ficar pronto

Essa é uma ideia fundamental.

Não devemos pensar:

Requisitos → desenvolvimento → software pronto → teste

Podemos testar desde cedo:

Requisitos → revisão/teste

Projeto → revisão/teste

Código → teste

Integração → teste

Sistema → teste

16. Shift-left

Shift-left significa **realizar atividades de teste mais cedo no processo de desenvolvimento**.

Exemplo:

Em vez de esperar o software ficar pronto para descobrir um requisito errado:

revisamos o requisito antes.

Fluxo:

problema encontrado cedo

→ correção mais simples

→ menos retrabalho.

17. TDD × ATDD × BDD

Não confunda.

TDD

Test-Driven Development

O desenvolvimento é orientado por testes.

Simplificando:

teste → **código** → **refatoração**

ATDD

Acceptance Test-Driven Development

Os testes de aceitação são definidos com base nos critérios de aceitação **antes da implementação**.

Fluxo:

História

↓

Discussão

↓

Critérios de aceitação

↓

Casos de teste

↓

Implementação

↓

Execução

BDD

Behavior-Driven Development

Foca no comportamento esperado do sistema, utilizando uma linguagem compreensível para diferentes participantes.

18. DevOps

DevOps aproxima desenvolvimento e operações.

A ideia inclui:

- colaboração;
- integração;
- entrega frequente;
- automação;
- feedback rápido.

Os testes fazem parte desse fluxo.

19. Níveis de teste

Os níveis representam **onde no sistema o teste está sendo realizado**.

Os principais são:

Teste de componente

Testa uma unidade/componente individual.

Teste de integração

Testa a interação entre componentes ou sistemas.

Teste de sistema

Avalia o sistema como um todo.

Teste de aceitação

Avalia se o sistema atende às necessidades e critérios de aceitação.

Teste de integração de sistema

Avalia interações entre sistemas.

20. Tipos de teste

Aqui a pergunta é diferente.

Nível = onde estamos testando?

Tipo = o que estamos avaliando?

Exemplos:

Teste funcional

Verifica funções do sistema.

Exemplo:

Ao clicar em “Entrar”, o usuário deve conseguir acessar sua conta com credenciais válidas.

Teste não funcional

Avalia características diferentes da função propriamente dita.

Exemplos:

- desempenho;
 - usabilidade;
 - segurança;
 - compatibilidade.
-

21. Teste de confirmação × regressão

Teste de confirmação

Pergunta:

“O defeito corrigido realmente foi corrigido?”

Exemplo:

Um login tinha um defeito.

O desenvolvedor corrige.

Executamos novamente o teste relacionado àquele defeito.

Isso é **confirmação**.

Teste de regressão

Pergunta:

“A mudança que fizemos quebrou alguma coisa que já funcionava?”

Depois da correção:

corrige A

↓

testa A

↓

testa B, C, D...

↓

verifica se outras funcionalidades continuam funcionando.

Isso é **regressão**.

Decore:

Confirmação → verificar a correção

Regressão → procurar efeitos colaterais

22. Teste de manutenção

Acontece quando um software já existente sofre mudanças ou precisa continuar sendo utilizado em novas condições.

Pode ocorrer por:

- correções;
- mudanças no ambiente;
- novas funcionalidades;
- migração;
- aposentadoria do sistema.

A manutenção pode exigir análise de impacto e testes de regressão.

CAPÍTULO 3 — TESTE ESTÁTICO

23. O que é teste estático?

Teste estático é realizado **sem executar o software**.

Pode envolver:

- revisão de requisitos;
- revisão de código;
- revisão de modelos;
- análise estática.

Já o teste dinâmico envolve executar o software.

Exemplo

Você recebe um requisito:

“O usuário deve conseguir recuperar a senha.”

Antes de existir código, a equipe pode revisar esse requisito e encontrar problemas.

Isso é teste estático.

24. Benefícios do teste estático

Pode encontrar problemas:

- mais cedo;
- antes da execução;
- em requisitos;
- em projetos;
- em código;
- em documentação.

Isso ajuda a reduzir retrabalho.

25. Revisões

Existem diferentes tipos.

Revisão informal

Menos formalidade.

Pode ser simplesmente alguém pedindo para outra pessoa revisar um documento.

Walkthrough

O **autor conduz a apresentação** do produto para os participantes.

Essa palavra é importante:

Walkthrough → **autor conduz**.

Revisão técnica

Envolve participantes tecnicamente qualificados e um moderador.

Inspeção

É a revisão mais formal.

Possui processo estruturado e busca encontrar o maior número possível de anomalias.

26. Processo de revisão

Uma sequência importante:

1. Planejamento
 2. Início da revisão
 3. Revisão individual
 4. Comunicação/análise dos problemas
 5. Correção e relatório
-

CAPÍTULO 4 — ANÁLISE E MODELAGEM DE TESTE

Esse é um dos capítulos mais importantes da prova.

O foco deixa de ser apenas:

“O que é essa técnica?”

e passa a ser:

“Consigo aplicar essa técnica?”

Os simulados possuem questões K3 sobre as técnicas de caixa-preta.

27. Três famílias de técnicas

Caixa-preta

Baseada no comportamento/especificação.

Não precisamos conhecer o código interno.

Pergunta:

“O que o sistema deveria fazer?”

Caixa-branca

Baseada na estrutura interna do software.

Pergunta:

“Quais partes do código foram executadas?”

Baseada em experiência

Utiliza:

- conhecimento;
- experiência;
- intuição;
- conhecimento de defeitos anteriores.

28. Particionamento de Equivalência — EP

A ideia é dividir as entradas em **grupos equivalentes**.

Exemplo:

Sistema aceita idade de:

18 até 60 anos

Podemos criar:

Partição	Valores
----------	---------

inválida	menor que 18
----------	--------------

válida	18 até 60
--------	-----------

inválida	maior que 60
----------	--------------

Não precisamos testar todas as idades.

Podemos escolher representantes:

- 17;
- 30;
- 61.

A ideia é que os valores dentro da mesma partição devem ser tratados de maneira semelhante.

29. BVA — Análise de Valor de Limite

BVA olha principalmente para **as fronteiras das partições**.

Se o sistema aceita:

18 até 60

Os limites são:

18 e 60

Testamos valores próximos:

17, 18, 19

e

59, 60, 61

Diferença:

EP → grupos

BVA → limites

30. Tabela de decisão

Usamos quando o comportamento depende de **combinações de condições**.

Exemplo:

Para conseguir um desconto:

- cliente é membro?
- compra ultrapassa R\$ 100?

Cada combinação pode produzir uma ação diferente.

Membro	> R\$100	Descont
--------	----------	---------

o

Não	Não	Não
-----	-----	-----

Não Sim Não

Sim Não Sim

Sim Sim Sim

Palavra-chave:

Combinações de condições → tabela de decisão

31. Transição de estado

Usamos quando o comportamento depende do **estado atual** e de um **evento**.

Exemplo de conta:

Ativa

↓ senha incorreta

Tentativa 1

↓ senha incorreta

Tentativa 2

↓ senha incorreta

Bloqueada

Aqui temos:

- estados;
- eventos;
- transições;
- ações/resultados.

Palavra-chave:

Estados + eventos → transição de estado

32. Caixa-branca: instrução × ramificação

Cobertura de instrução

Verifica quais instruções do código foram executadas.

Pergunta:

“Quantas instruções foram executadas?”

Cobertura de ramificação

Verifica os caminhos/decisões.

Exemplo:

se idade >= 18

 aprovado

senão

 reprovado

Precisamos considerar:

- verdadeiro;
- falso.

Decore:

Instrução → **statements**

Ramificação → **branches/decisões**

33. Técnicas baseadas em experiência

Suposição de erro

O tester utiliza conhecimento sobre erros que costumam acontecer.

Exemplo:

Campos de entrada frequentemente aceitam valores inválidos.

Então o tester tenta:

- campo vazio;
 - caracteres especiais;
 - valores muito grandes;
 - formatos inesperados.
-

Teste exploratório

O teste e o aprendizado acontecem de forma simultânea.

O tester:

explora

- aprende
- cria novas ideias
- testa
- encontra novas informações
- continua explorando.

É útil quando existe pouco tempo ou pouca documentação, por exemplo.

Checklist

O tester utiliza uma lista de itens que devem ser verificados.

Exemplo:

- login;
 - logout;
 - senha inválida;
 - recuperação de senha;
 - mensagens de erro;
 - permissões.
-

34. História de usuário

Uma história de usuário descreve uma necessidade do ponto de vista do usuário.

Estrutura comum:

Como **[tipo de usuário]**, quero **[objetivo]**, para **[benefício]**.

Exemplo:

Como cliente, quero recuperar minha senha para conseguir acessar minha conta novamente.

35. INVEST

Características de uma boa história de usuário:

I — Independent

Independente

N — Negotiable

Negociável

V — Valuable

Valiosa

E — Estimable

Estimável

S — Small

Pequena

T — Testable

Testável

36. Critérios de aceitação

São condições que precisam ser atendidas para que a implementação seja aceita.

Exemplo:

O usuário deve conseguir recuperar a senha utilizando seu e-mail cadastrado.

Os critérios ajudam a:

- definir o escopo;
 - criar testes;
 - alinhar expectativas;
 - planejar;
 - estimar.
-

37. Critérios orientados a cenário

Um formato comum:

Dado

→ situação inicial

Quando

→ ação

Então

→ resultado esperado

Exemplo:

Dado que o usuário possui uma conta
Quando informar e-mail e senha corretos
Então deve acessar o sistema.

Palavra-chave:

Dado / Quando / Então → cenário

38. ATDD

No ATDD, os testes de aceitação são definidos **antes da implementação**, com base nos critérios de aceitação.

Fluxo:

História

↓

Workshop/discussão

↓

Critérios de aceitação

↓

Casos de teste

↓

Implementação

↓

Execução

CAPÍTULO 5 — GERENCIAMENTO DAS ATIVIDADES DE TESTE

Esse é o outro grande capítulo da prova.

Aqui você deixa de perguntar apenas:

“Como testar?”

e passa a perguntar:

“Como organizar e administrar todo o trabalho de teste?”

39. Plano de teste

O plano responde:

“Como vamos realizar os testes?”

Pode conter:

- objetivos;
- escopo;
- abordagem;
- recursos;
- cronograma;
- responsabilidades;
- riscos;
- ambiente;
- comunicação;
- métricas;
- orçamento.

40. Release planning × Iteration planning

Release planning

Olha para uma **liberação do produto**.

Considera:

- backlog;
- histórias;
- riscos;
- esforço;
- abordagem de teste.

Iteration planning

Olha para uma **iteração específica**.

Considera:

- histórias da iteração;
 - tarefas;
 - testabilidade;
 - esforço;
 - riscos;
 - aspectos funcionais e não funcionais.
-

41. Critérios de entrada × saída

Entrada

Pergunta:

“O que precisa estar pronto para começar?”

Exemplos:

- ambiente disponível;
- dados disponíveis;
- casos de teste prontos;

- recursos disponíveis.

Saída

Pergunta:

“O que precisa ser alcançado para terminar?”

Exemplos:

- testes planejados executados;
 - cobertura mínima alcançada;
 - defeitos relatados.
-

42. DoR × DoD

No desenvolvimento ágil:

DoR — Definition of Ready

Define quando um item está **pronto para começar**.

Relaciona-se aos critérios de entrada.

DoD — Definition of Done

Define quando um item pode ser considerado **concluído**.

Relaciona-se aos critérios de saída.

Decore:

Ready → começar

Done → terminar

43. Estimativa de esforço

Precisamos estimar:

Quanto trabalho será necessário para realizar os testes?

Uma forma é utilizar proporções históricas.

Exemplo:

Projetos anteriores indicam:

teste = 10% do esforço de desenvolvimento.

Se desenvolvimento exige:

800 horas

Então:

$800 \times 10\% = 80$ horas de teste

Esse tipo de aplicação aparece como K3 nos simulados.

44. Estimativa de três pontos

Considera três estimativas:

O — Otimista

Se tudo correr bem.

M — Mais provável

Cenário mais esperado.

P — Pessimista

Se ocorrerem problemas.

Uma fórmula comum:

$E = (O + 4M + P) / 6$

Exemplo:

O = 10

M = 20

P = 40

Então:

$E = (10 + 4 \times 20 + 40) / 6$

$$E = 130 / 6$$

$$E \approx 21,7$$

45. Priorização

Nem todos os testes possuem a mesma importância.

A prioridade pode considerar:

- risco;
- importância da funcionalidade;
- impacto;
- probabilidade;
- dependências;
- valor para o negócio.

Imagine:

Teste A → pagamento

Teste B → trocar foto de perfil

Se o pagamento for uma funcionalidade crítica, ele pode receber prioridade maior.

46. Dependências

Às vezes um teste depende de outro.

Exemplo:

Não consigo testar “realizar pagamento” antes de testar/configurar “login”.

Então existe uma dependência.

Quando houver dependências, precisamos considerar a ordem de execução.

47. Pirâmide de testes

A pirâmide representa uma distribuição de diferentes níveis de automação/testes.

De forma geral:

Base

Muitos testes menores e mais rápidos.

Meio

Quantidade intermediária.

Topo

Poucos testes mais amplos, lentos ou caros.

A ideia é evitar depender excessivamente de testes grandes e caros.

48. Quadrantes de teste

Os quadrantes ajudam a organizar os testes conforme seu objetivo e perspectiva.

Eles podem ajudar a diferenciar:

- testes voltados ao negócio;
- testes voltados à tecnologia;
- testes que apoiam o desenvolvimento;
- testes que avaliam o produto.

O importante para a prova é **reconhecer o propósito do teste e relacioná-lo ao quadrante adequado**, em vez de simplesmente decorar quatro nomes.

49. Risco

Risco é uma situação incerta que pode causar consequências.

Dois conceitos são especialmente importantes:

Probabilidade

Qual a chance de acontecer?

Impacto

Se acontecer, qual será a consequência?

Uma forma simples de pensar:

Risco $\approx$ probabilidade $\times$ impacto

50. Risco de projeto $\times$ risco de produto

Risco de projeto

Está relacionado ao próprio projeto.

Exemplos:

- falta de pessoas;
- atraso;
- orçamento;
- ferramentas;
- cronograma.

Risco de produto

Está relacionado ao **produto/software**.

Exemplos:

- funcionalidade incorreta;
- baixa performance;
- problema de segurança;
- dificuldade de uso.

Decore:

Projeto $\rightarrow$ conseguir entregar

Produto $\rightarrow$ qualidade do que está sendo entregue

51. Teste baseado em risco

Quando existem muitos riscos, os testes podem ser direcionados para as áreas de maior risco.

Exemplo:

Um sistema bancário possui:

- trocar foto;
- consultar saldo;
- realizar transferência.

Se transferência apresenta grande impacto caso falhe, ela pode receber atenção especial.

52. Monitoramento e controle

Monitoramento

Acompanhar:

“Como está o trabalho?”

Exemplos:

- quantos testes executados;
- quantos passaram;
- quantos falharam;
- quantos defeitos encontrados.

Controle

Tomar decisões com base nas informações.

Exemplo:

Percebemos que os testes estão atrasados. Precisamos reorganizar o cronograma.

53. Métricas

Métrica é uma medida utilizada para obter informações sobre o teste.

Exemplos:

- quantidade de testes executados;
- testes aprovados;
- testes falhos;
- defeitos encontrados;
- cobertura;
- esforço gasto.

Não confunda:

Métrica → medida

Relatório → comunicação organizada das informações

54. Relatórios

O relatório apresenta informações relevantes para stakeholders.

Pode comunicar:

- progresso;
- resultados;
- riscos;
- defeitos;
- cobertura;
- situação atual;
- conclusão.

A comunicação deve ser clara e adequada ao público.

55. Gerenciamento de configuração

Configuration Management ajuda a controlar quais versões dos produtos estão sendo utilizados.

Imagine:

O tester executou um teste na versão:

2.4

Mas o desenvolvedor corrigiu o sistema e publicou:

2.5

Precisamos saber exatamente qual versão foi testada.

Isso evita confusão e ajuda na rastreabilidade.

56. Relatório de defeito

Um relatório de defeito precisa permitir que outras pessoas entendam e reproduzam o problema.

Pode conter:

- identificador;
 - título;
 - ambiente;
 - versão;
 - passos para reproduzir;
 - resultado esperado;
 - resultado obtido;
 - evidências;
 - severidade;
 - prioridade.
-

57. Severidade × prioridade

Severidade

Pergunta:

“Quanto o problema impacta o sistema?”

Exemplo:

Um erro impede todos os usuários de realizar pagamentos.

Pode possuir alta severidade.

Prioridade

Pergunta:

“Quão urgente é corrigir isso?”

Um problema pode ter:

- alta severidade e alta prioridade;
- baixa severidade e alta prioridade;
- alta severidade e prioridade diferente, dependendo do contexto.

Não são a mesma coisa.

CAPÍTULO 6 — FERRAMENTAS

Esse capítulo é menor, mas também faz parte da prova.

58. Ferramentas de teste

Ferramentas podem apoiar diferentes atividades.

Podem auxiliar em:

- gerenciamento de testes;
 - execução;
 - automação;
 - gestão de defeitos;
 - desempenho;
 - análise estática;
 - CI/CD;
 - preparação de dados;
 - monitoramento.
-

59. Benefícios da automação

A automação pode:

- executar testes repetitivos;
- aumentar velocidade;
- facilitar execução frequente;

- produzir resultados consistentes;
 - reduzir trabalho manual repetitivo.
-

60. Riscos da automação

Automação não resolve tudo.

Pode trazer:

- custo inicial;
- manutenção dos scripts;
- testes automatizados frágeis;
- falsa sensação de segurança;
- problemas com ferramentas;
- necessidade de conhecimento técnico.

Portanto:

Automatizar não significa automaticamente melhorar tudo.

O CTFL pede justamente que o candidato entenda **benefícios e riscos da automação**.

MAPA PARA NÃO CONFUNDIR OS CONCEITOS

Teste

Encontrar defeitos e avaliar qualidade.

Depuração

Encontrar a causa e corrigir o problema.

Verificação

Estamos construindo corretamente?

Validação

Estamos construindo o produto certo?

Estático

Sem executar o software.

Dinâmico

Executando o software.

Confirmação

A correção funcionou?

Regressão

A mudança quebrou outra coisa?

EP

Dividir em grupos.

BVA

Testar limites.

Tabela de decisão

Combinações de condições.

Transição de estado

Estados + eventos.

Caixa-preta

Comportamento/especificação.

Caixa-branca

Estrutura interna.

Experiência

Conhecimento do tester.

Entrada

Pode começar?

Saída

Pode terminar?

DoR

Está pronto para começar?

DoD

Está concluído?

Risco de projeto

Problemas para realizar o projeto.

Risco de produto

Problemas relacionados à qualidade do produto.

Severidade

Tamanho/impacto do problema.

Prioridade

Urgência para corrigir.

COMO RESOLVER UMA QUESTÃO DA PROVA

Quando aparecer um cenário, **não procure imediatamente uma definição decorada.**

Faça estas quatro perguntas:

1. O que a questão está descrevendo?

É:

- requisito?
- código?
- entrada?
- limite?
- estado?
- risco?
- defeito?
- planejamento?

2. Qual é a palavra-chave?

Exemplos:

“grupos de valores” → **EP**

“limite” → **BVA**

“combinação de condições” → **tabela de decisão**

“estado/evento” → **transição de estado**

“corrigir defeito” → **depuração**

“verificar se a correção funcionou” → **confirmação**

“verificar se outra funcionalidade que já funcionava quebrou” → **regressão**

3. Qual conceito responde exatamente ao problema?

Não escolha porque a alternativa “parece familiar”.

4. Aplique o conceito ao cenário.

Essa última etapa é especialmente importante porque os simulados não cobram somente memorização.

O FLUXO COMPLETO DO CTFL

Se você entender esta sequência, começa a enxergar a matéria inteira como uma coisa só:

Requisito



Análise

O que precisa ser testado?



Condição de teste



Técnica de teste

EP? BVA? tabela? estado?



Caso de teste



Implementação



Execução



Resultado



Falha?



Defeito



Relatório



Depuração

↓

Correção

↓

Teste de confirmação

↓

Teste de regressão

↓

Métricas e relatórios

↓

Decisão

↓

Conclusão

Esse fluxo reúne justamente as atividades e os produtos de trabalho que aparecem ao longo dos seis capítulos. O material do curso também apresenta essa lógica geral.

O QUE VOCÊ PRECISA DOMINAR PARA A PROVA

Se você nunca leu o livro, eu estudaria nesta ordem:

Primeiro — entenda

- o que é teste;
- teste × depuração;
- erro × defeito × falha;
- verificação × validação;
- 7 princípios;
- Testware;
- rastreabilidade.

Depois — entenda o processo

- SDLC;

- shift-left;
- DevOps;
- TDD/ATDD/BDD;
- níveis de teste;
- tipos de teste;
- confirmação × regressão;
- manutenção.

Depois — aprenda estático

- teste estático;
- revisão;
- walkthrough;
- revisão técnica;
- inspeção;
- papéis;
- processo de revisão.

Depois — PRATIQUE

Principalmente:

- EP;
- BVA;
- tabela de decisão;
- transição de estado;
- cobertura de instrução;
- cobertura de ramificação.

Depois — colaboração

- história de usuário;
- INVEST;
- critérios de aceitação;
- ATDD.

Depois — gerenciamento

- plano de teste;
- entrada/saída;
- DoR/DoD;
- estimativa;
- três pontos;
- priorização;
- dependências;
- pirâmide;
- quadrantes;
- riscos;

- métricas;
- relatórios;
- configuração;
- defeitos;
- severidade/prioridade.

Por último — ferramentas

- categorias;
- benefícios da automação;
- riscos da automação.

O próprio syllabus distribui **390 minutos para o Capítulo 4 e 335 minutos para o Capítulo 5**, muito mais do que os 20 minutos do Capítulo 6, então faz sentido dedicar muito mais tempo a técnicas e gerenciamento.

Esse é o resumo que eu usaria como “primeira leitura”, enquanto o conteúdo anterior funciona melhor como **revisão depois que você já entendeu a matéria**.